

Chapter 3 All previous Questions

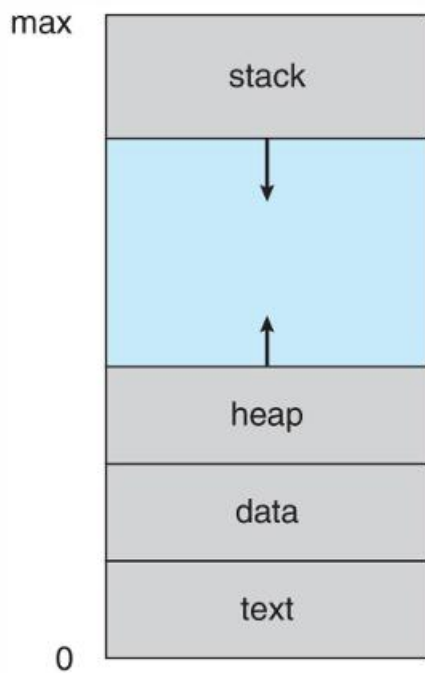
2. (a) Identify the separate components of a process and illustrate how they are represented and scheduled in an operating system. **(3)**

Separate Components of a Process and Their Representation & Scheduling in an Operating System

A **process** is a program in execution. An operating system divides a process into several components so that it can manage execution efficiently.

Main Components of a Process

1. **Program Code (Text Section)**
 - Contains the executable instructions of the program.
 - This part does not change during execution.
2. **Data Section**
 - Stores global and static variables.
 - Exists throughout the lifetime of the process.
3. **Heap**
 - Used for dynamic memory allocation during runtime.
 - Memory is allocated and deallocated as needed.
4. **Stack**
 - Stores function calls, local variables, and return addresses.
 - Operates in LIFO (Last In First Out) order.
5. **Program Counter (PC)**
 - Holds the address of the next instruction to execute.
6. **CPU Registers**
 - Store temporary data, process state, and execution information.



Representation of a Process in Operating System

The operating system represents each process using a **Process Control Block (PCB)**.

Process Control Block (PCB)

PCB is a data structure that contains all information related to a process.

Contents of PCB

- Process ID (PID)
- Process State
- Program Counter
- CPU Registers
- Memory Management Information
- CPU Scheduling Information
- I/O Status Information

Simple Diagram of PCB



Process Scheduling in Operating System

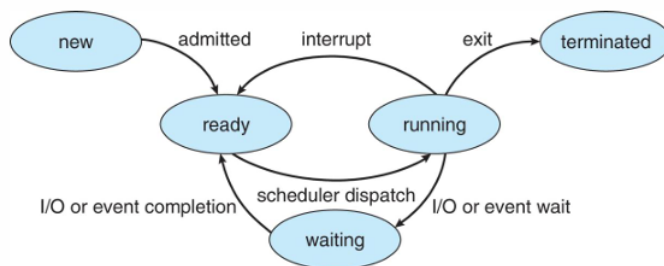
The operating system schedules processes using different queues and schedulers.

Process States

A process generally moves through the following states:

1. **New** – Process is being created.
2. **Ready** – Waiting for CPU allocation.
3. **Running** – Instructions are being executed.
4. **Waiting/Blocked** – Waiting for I/O or an event.
5. **Terminated** – Process execution finished.

Diagram of Process State



১. New (নতুন)

যখন একটি প্রোগ্রাম কেবল শুরু করা হয়েছে কিন্তু এখনো মেমরিতে জায়গা পায়নি, তখন একে বলা হয় **New** স্টেট। এখান থেকে অনুমতি পেলে (**admitted**) সে পরের ধাপে যায়।

২. Ready (প্রস্তুত)

প্রসেসটি এখন মেইন মেমরিতে (RAM) আছে এবং কাজ শুরু করার জন্য পুরোপুরি তৈরি। সে শুধু অপেক্ষা করছে প্রসেসরের (CPU) সিরিয়াল পাওয়ার জন্য।

৩. Running (চলমান)

যখন CPU খালি হয়, তখন **Scheduler** প্রসেসটিকে বেছে নেয় (**scheduler dispatch**)। এখন প্রসেসটির কাজ বাস্তবে শুরু হয়। এটিই হলো **Running** স্টেট। এখান থেকে তিনটি পথ হতে পারে:

- **Interrupt:** যদি কোনো জরুরি কাজ চলে আসে, তবে অপারেটিং সিস্টেম রানিং প্রসেসকে থামিয়ে আবার **Ready** স্টেটে পাঠিয়ে দেয়।
- **I/O or event wait:** যদি প্রসেসটির কোনো ইনপুট লাগে (যেমন: আপনার কিবোর্ড টাইপের অপেক্ষা) বা অন্য কোনো ফাইল লোড করতে হয়, তবে সে **Waiting** স্টেটে চলে যায়।
- **Exit:** কাজ শেষ হয়ে গেলে সে সরাসরি শেষের ধাপে চলে যায়।

৪. Waiting (অপেক্ষমাণ)

প্রসেসটি এখানে বসে থাকে যতক্ষণ না তার প্রয়োজনীয় ইনপুট/আউটপুট কাজ শেষ হচ্ছে। কাজ শেষ হয়ে গেলে সে আবার **Ready** স্টেটে ফিরে যায় (সরাসরি রানিং-এ যেতে পারে না, তাকে আবার সিরিয়াল ধরতে হয়)।

৫. Terminated (সমাপ্ত)

যখন প্রসেসটির সব কাজ সফলভাবে শেষ হয়, তখন সে **Terminated** স্টেটে চলে যায়। অর্থাৎ অপারেটিং সিস্টেম তাকে মেমরি থেকে মুছে ফেলে।

How Processes Are Scheduled

1. Ready Queue

- Contains processes waiting for CPU execution.

2. CPU Scheduler

- Selects one process from the ready queue.

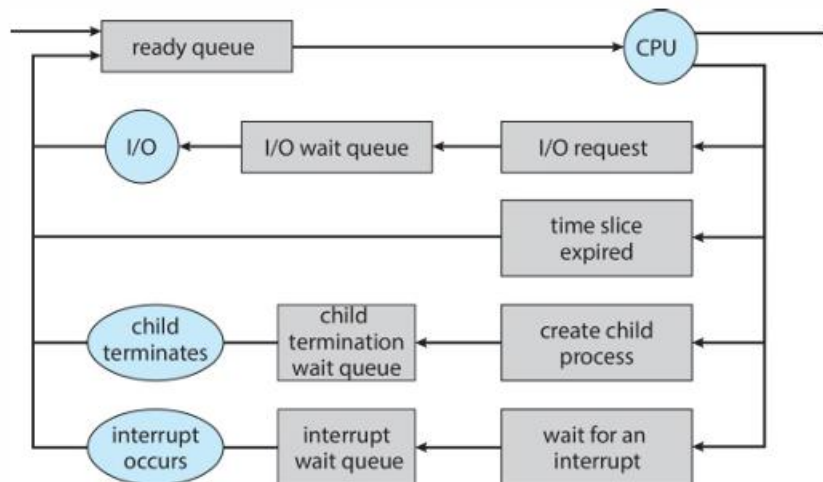
3. Dispatcher

- Gives control of CPU to the selected process.

Common Scheduling Algorithms

- FCFS (First Come First Serve)
- SJF (Shortest Job First)
- Round Robin
- Priority Scheduling

Representation of Process Scheduling



১. মেইন গेट: Ready Queue

যেকোনো নতুন প্রসেস প্রথমে **Ready Queue**-তে এসে লাইন ধরে। যখন CPU খালি হয়, তখন এই লাইন থেকে একটি প্রসেসকে কাজের জন্য নেওয়া হয়।

২. CPU-তে কাজ চলাকালীন ৪টি পরিস্থিতি

CPU যখন কোনো প্রসেসকে প্রসেস করছে, তখন হঠাৎ ৪টি কারণে সেই প্রসেসটি CPU থেকে বের হয়ে যেতে পারে:

- **I/O Request:** প্রসেসটির যদি কোনো ইনপুট (যেমন: স্ক্যানার বা কিবোর্ড) লাগে, তবে সে **I/O wait queue**-তে গিয়ে বসে থাকে। কাজ শেষ হলে সে আবার শুরুর **Ready Queue**-তে গিয়ে সিরিয়াল ধরে।

- **Time Slice Expired:** প্রতিটি প্রসেসকে CPU-তে খুব অল্প সময় দেওয়া হয়। সেই সময় (Time Slice) শেষ হয়ে গেলে প্রসেসটি কাজ শেষ না হলেও বাধ্য হয়ে আবার **Ready Queue**-তে ফিরে যায়।
 - **Create Child Process:** যদি একটি প্রসেস কাজ চলাকালীন অন্য কোনো নতুন প্রসেস (Child) তৈরি করে, তবে সে নিজে **Child termination wait queue**-তে অপেক্ষা করে। যখন বাচ্চা প্রসেসটি কাজ শেষ করে, তখন মেইন প্রসেসটি আবার **Ready Queue**-তে ফেরত যায়।
 - **Wait for an Interrupt:** যদি কোনো এক্সটারনাল সিগন্যাল বা ইন্টারাপ্ট আসে, তবে প্রসেসটি **Interrupt wait queue**-তে চলে যায়। ইন্টারাপ্ট চলে গেলে সে আবার লাইনে দাঁড়ায়।
-

Conclusion

A process consists of code, data, heap, stack, and CPU execution information. The operating system represents processes using the PCB and manages their execution through scheduling mechanisms and process states.

(b) Define remote procedure calls. Describe and contrast interprocess communication using shared memory and message passing. (4)

Remote Procedure Call (RPC)

A **Remote Procedure Call (RPC)** is a communication mechanism that allows a program to execute a procedure or function on another computer or remote system as if it were a local function call.

Remote Procedure Call (RPC) হলো এমন একটি communication system যেখানে একটি program অন্য কম্পিউটার বা remote system-এর কোনো function/procedure কে এমনভাবে call করতে পারে যেন সেটি নিজের computer-এর local function।

সহজভাবে বললে,

এক কম্পিউটার অন্য কম্পিউটারের function ব্যবহার করে network-এর মাধ্যমে কাজ করে।

RPC-এর একটি সহজ example হলো:

ধরো, তোমার মোবাইলের একটি app weather information দেখায়।
তুমি যখন app-এ “Get Weather” চাপ দাও, তখন app তোমার phone থেকে remote server-এ request পাঠায়।

Server তখন:

1. Weather data collect করে
2. সেই data process করে
3. Result আবার তোমার app-এ পাঠায়

তোমার কাছে মনে হয় যেন app-এর ভেতরের কোনো local function কাজ করছে, কিন্তু আসলে কাজটি remote server-এ হয়েছে। এটিই RPC।

Working Principle of RPC

1. The client sends a request to the remote server.
2. The RPC system packages the request (marshalling).
3. The request is transmitted through the network.
4. The server executes the requested procedure.
5. The result is sent back to the client.

Advantages

- Simplifies distributed computing.
- Hides network communication details.
- Makes remote calls appear like local function calls.

Interprocess Communication (IPC)

IPC allows processes to exchange data and synchronize their actions. Two common IPC methods are:

1. Shared Memory
2. Message Passing

Shared Memory

In **shared memory**, two or more processes share a common memory area.

Working Process

- The operating system creates a shared memory segment.
- Multiple processes access and modify the shared area directly.

Advantages

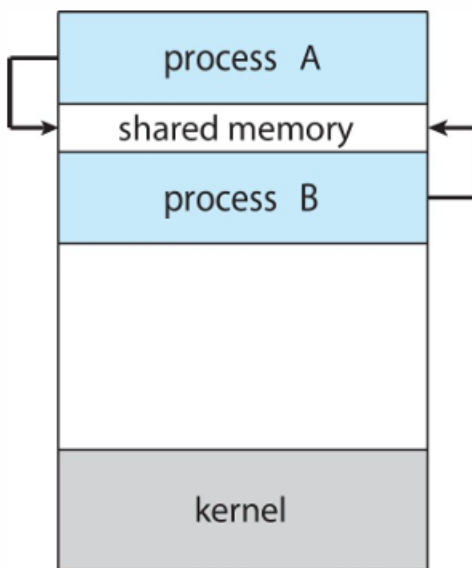
- Very fast communication.
- Suitable for large amounts of data.

Disadvantages

- Requires synchronization mechanisms like semaphores or mutexes.
- More complex to manage.

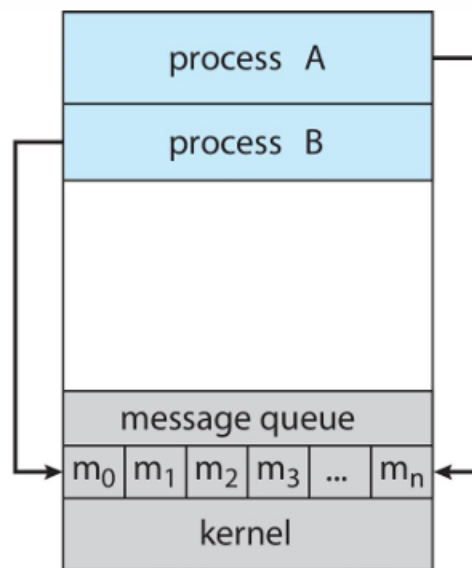
Communications Models

(a) Shared memory.



(a)

(b) Message passing.



(b)

Message Passing

In **message passing**, processes communicate by sending and receiving messages through the operating system.

Working Process

- One process sends a message.
- Another process receives the message.

Advantages

- Easier to implement.
- No shared data conflicts.

Disadvantages

- Slower than shared memory.
- Communication overhead is higher.

Comparison Between Shared Memory and Message Passing

Feature	Shared Memory	Message Passing
Speed	Faster	Slower
Data Transfer	Direct memory access	Through OS messages
Synchronization	Required	Usually not required
Complexity	More complex	Simpler
Suitable For	Large data sharing	Small data communication

Conclusion

RPC enables a process to call procedures on remote systems. Shared memory and message passing are two important IPC methods. Shared memory provides high-speed communication, while message passing offers simpler and safer communication between processes.

4.(c) Why applications are operating system specific? Identify the separate components of a process and illustrate how they are represented and scheduled in an operating system. (3)

Applications are **operating system specific** because different operating systems provide **different environments, system calls, hardware management methods, and file systems**. An application is developed to work with the services and rules of a particular operating system.

Example

- Microsoft Word for Windows cannot run directly on Linux.
 - Android apps cannot run directly on Windows without special software.
-

Conclusion

Applications are OS specific because operating systems differ in system calls, file systems, hardware control, executable formats, and user interfaces. Therefore, software must be designed and compiled according to the target operating system.

4.(d) Define process. Describe how processes are created and terminated in an operating system, including developing programs using the appropriate system calls that perform these operations. (4)

Process Definition

A process is a program in execution.

It includes:

- Program code
- Data
- Stack
- Heap
- Program counter

- CPU registers

A process is an active entity that performs tasks under the control of the operating system.

Process Creation in Operating System

A new process is created when:

- A user starts a program
- Another process creates a child process
- The system starts a service or background task

Steps of Process Creation

1. Assign a unique Process ID (PID)
2. Create Process Control Block (PCB)
3. Allocate memory and resources
4. Load program code into memory
5. Place the process in the ready queue

System Calls for Process Creation

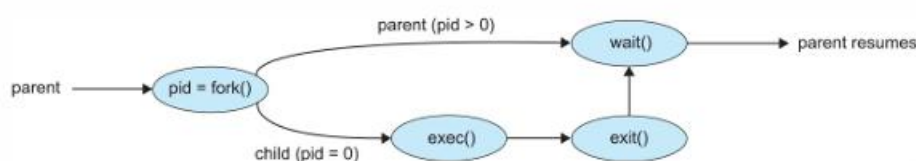
In UNIX/Linux systems, the main system calls are:

1. `fork()`

- Creates a new child process.
- The child process is a copy of the parent process.

2. `exec()`

- Replaces the current process memory with a new program.



১. প্রসেস তৈরি (`fork`)

সবকিছুর শুরু হয় **parent** প্রসেস থেকে। যখন একটি নতুন প্রসেস দরকার হয়, তখন `fork()` ফাংশনটি কল করা হয়।

- এই `fork()` কল করার মাধ্যমে মেইন প্রসেসটি নিজের একটি ছবছ কপি বা **Child Process** তৈরি করে।
- এখানে একটি মজার ব্যাপার ঘটে: `fork()` ফাংশনটি দুই ধরনের ভ্যালু রিটার্ন করে।
 - যদি রিটার্ন ভ্যালু `০` হয় (`pid = 0`), তবে কম্পিউটার বুঝে নেয় এটা **Child** প্রসেস।
 - যদি রিটার্ন ভ্যালু `০` এর বেশি হয় (`pid > 0`), তবে সেটা **Parent** প্রসেস।

২. চাইল্ড প্রসেসের কাজ (`exec` ও `exit`)

নিচের দিকের রাস্তাটি খেয়াল করো:

- `exec()`: চাইল্ড প্রসেস তৈরি হওয়ার পর সে সাধারণত `exec()` সিস্টেম কল ব্যবহার করে। এর কাজ হলো পুরনো প্রোগ্রামের জায়গায় নতুন কোনো প্রোগ্রাম লোড করে চালানো।
- `exit()`: যখন চাইল্ড প্রসেস তার কাজ শেষ করে ফেলে, তখন সে `exit()` কল করে নিজেকে শেষ করে দেয়।

৩. প্যারেন্ট প্রসেসের অপেক্ষা (`wait`)

উপরের দিকের রাস্তাটি দেখো:

- `wait()`: চাইল্ড যখন কাজ করছে, তখন প্যারেন্ট প্রসেস সাধারণত বসে থাকে না। সে `wait()` ফাংশন ব্যবহার করে অপেক্ষা করে যতক্ষণ না চাইল্ড প্রসেস তার কাজ শেষ (`exit`) করছে।
- চাইল্ড যখন মারা যায় বা কাজ শেষ করে, তখন সে প্যারেন্টকে একটি সিগন্যাল পাঠায়।

৪. সমাপ্তি (`parent resumes`)

- চাইল্ড প্রসেস কাজ শেষ করে বিদায় নিলে প্যারেন্ট প্রসেস তার ‘অপেক্ষা’ (`wait`) শেষ করে এবং বাকি কাজগুলো করা শুরু করে (`parent resumes`)।

Example Program for Process Creation(NTK

```
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid;

    pid = fork();

    if(pid == 0) {
        printf("Child Process Running\n");
    }
    else {
        printf("Parent Process Running\n");
    }

    return 0;
}
```

Output

```
Parent Process Running
Child Process Running
```

Process Termination in Operating System

A process is terminated when:

- It finishes execution normally
- An error occurs
- Parent process terminates it
- Operating system stops it

Steps of Process Termination

1. Process finishes execution
 2. OS releases allocated resources
 3. PCB is removed
 4. Process state changes to terminated
-

System Calls for Process Termination

1. `exit()`

- Ends the current process normally.

2. `kill()`

- Terminates another process using its PID.

Example Program for Process Termination

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    printf("Process is running\n");

    exit(0);

    printf("This line will not execute");

    return 0;
}
```

Conclusion

A process is an executing program managed by the operating system. Processes are created using system calls like `fork()` and `exec()`, and terminated using `exit()` or `kill()`. The operating system handles memory allocation, scheduling, and resource management during the process lifecycle.

5. (a) What is process control block (PCB)? Describe and contrast inter-process communication using shared memory and message passing. (3)

Process Control Block (PCB)

PCB is a data structure that contains all information related to a process.

Contents of PCB

- Process ID (PID)
- Process State

- Program Counter
- CPU Registers
- Memory Management Information
- CPU Scheduling Information
- I/O Status Information

Simple Diagram of PCB



Describe the actions taken by a kernel to context-switch between processes. (Book's Exercise 3.8) ★★ [17-18 FINAL] [18-19 FINAL]

Context Switching Between Processes

A **context switch** is the process of saving the state of the currently running process and loading the state of another process so that CPU execution can be transferred.

It is performed by the **kernel** during process scheduling.

Actions Taken by the Kernel During Context Switch

When switching from one process to another, the kernel performs the following steps:

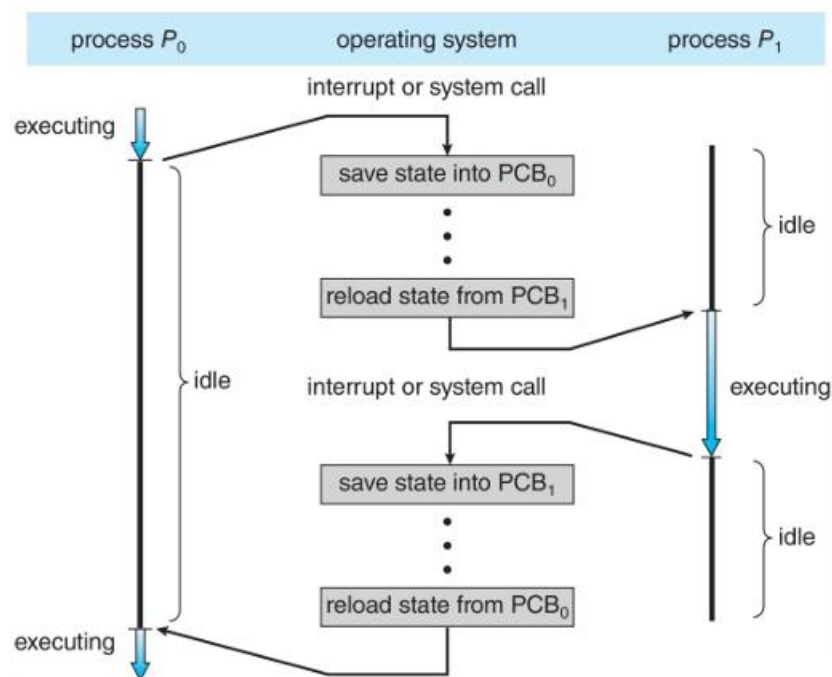
1. Save the Context of Current Process

The kernel saves the current process information **into its Process Control Block (PCB).**

Saved Information

- Program Counter (PC)
- CPU registers
- Process state
- Memory-management information
- Scheduling information

Diagram of Context Switching



2. Change Process State

- The current process state is changed:
 - Running $\rightarrow$ Ready, or
 - Running $\rightarrow$ Waiting

3. Select a New Process

- The CPU scheduler selects another process from the ready queue.

4. Load the Context of the Selected Process

The kernel loads the **saved information of the selected process from its PCB.**

Loaded Information

- CPU registers
- Program counter
- Memory information
- Process state

5. Update Process State

- The selected process state becomes:
 - Ready → Running

6. Resume Execution

- CPU starts executing the new process from the saved instruction point.

Important Notes

- **Context switching is pure overhead because no useful work is done during switching.**
- Frequent context switching reduces system performance.
- Hardware support helps make context switching faster.

Conclusion

During a context switch, the kernel saves the current process state into its PCB, selects another ready process, restores its saved state, and transfers CPU control to it. This mechanism allows multitasking in operating systems.

What is context switch? "Context switch time is pure overhead" - justify the statement. ★★
[12-13 FINAL] [13-14 FINAL]

“Context Switch Time is Pure Overhead” — Justification

A context switch does not perform any useful computation for user programs. During this time, the CPU only saves and restores process information.

Why It Is Called Pure Overhead

1. No Useful Work Done

- The CPU spends time switching processes instead of executing actual program instructions.

2. Consumes System Resources

- Memory access and CPU cycles are required to save and load contexts.

3. Reduces Performance

- Frequent context switching decreases overall system efficiency.

4. Cache and Pipeline Effects

- CPU cache and pipeline contents may become invalid, causing additional delays.

Example

Suppose Process A is running and Process B must execute:

- The OS pauses Process A.
- Saves its registers and state.
- Loads Process B's state.
- Starts Process B.

During this switching period, no user task is completed. Therefore, the time used is considered overhead.

Define cooperating process. ★ [19-20 FINAL]

A **cooperating process** is a process that can affect or be affected by other processes executing in the system.

These processes share data, resources, or information with other processes and often communicate through **Interprocess Communication (IPC)** mechanisms such as shared memory or message passing.

Cooperating Process কী?

Cooperating process হলো এমন process যা system-এর অন্য process-এর উপর প্রভাব ফেলতে পারে অথবা অন্য process দ্বারা প্রভাবিত হতে পারে।

এই process গুলো একে অপরের সাথে:

- data share করে,
- resource share করে,
- information exchange করে।

এরা সাধারণত IPC (Interprocess Communication) ব্যবহার করে communication করে, যেমন:

- Shared Memory
- Message Passing

Example

- A web browser process communicating with a download manager.
- Producer-consumer systems where one process produces data and another consumes it.

Conclusion

A cooperating process is a process that interacts with other processes by sharing information or resources to achieve efficient and coordinated execution.

Give an example of a situation in which ordinary pipes are more suitable than named pipes and an example of a situation in which named pipes are more suitable than ordinary pipes.
(Book's Exercise 3.14) ★ [18-19 FINAL]

Ordinary Pipes vs Named Pipes

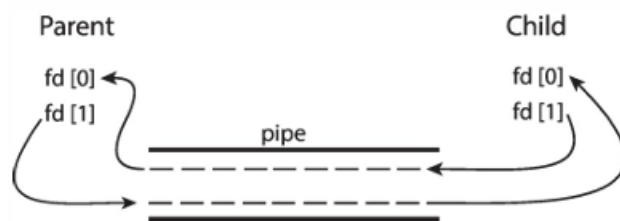
Ordinary Pipes

An **ordinary pipe** is used for communication between related processes, usually a parent and child process. It exists only while the processes are running.

Ordinary Pipe কী?

Ordinary pipe হলো এমন IPC method যা সাধারণত related process-এর মধ্যে communication-এর জন্য ব্যবহার হয়, যেমন parent process ও child process।

এটি process চলাকালীন সময় পর্যন্ত থাকে। Process শেষ হলে pipe-ও নষ্ট হয়ে যায়।



Suitable Situation for Ordinary Pipe

An ordinary pipe is more suitable when:

- A parent process creates a child process using `fork()`
- Temporary communication is needed between them

Example

In UNIX/Linux:

```
ls | grep ".txt"
```

Here:

- `ls` sends its output through an ordinary pipe
- `grep` receives the data

The two processes are related and the communication is temporary.

Named Pipes (FIFO)

A **named pipe** is a **special file** used for communication between unrelated processes. It remains in the file system until deleted.

Named Pipe (FIFO) কী?

Named pipe হলো special file-based IPC mechanism যা unrelated process-এর মধ্যে communication করতে পারে।

এটি file system-এ থাকে যতক্ষণ পর্যন্ত delete না করা হয়।

Suitable Situation for Named Pipe

A named pipe is more suitable when:

- Unrelated processes need communication
- Communication must continue independently of process creation

Named Pipe কখন বেশি Suitable?

যখন:

- Unrelated process-এর মধ্যে communication দরকার
- Process আলাদাভাবে start হলেও communication করতে হবে

Example

A chat application where:

- One process writes messages to the named pipe
- Another unrelated process reads the messages

Processes can communicate even if they were started separately.

Comparison

Feature	Ordinary Pipe	Named Pipe
Process Relationship	Related processes	Unrelated or related processes
Existence	Temporary	Persistent
Communication	Usually one-way	More flexible
File System Entry	No	Yes

Conclusion

Ordinary pipes are best for temporary communication between related processes, while named pipes are better for communication between unrelated processes that may run independently.

What are the benefits and the disadvantages of each of the following? Synchronous and asynchronous communication; Automatic and explicit buffering; Send by copy and send by reference; Fixed-sized and variable-sized messages. (Book's Exercise 3.17) ★ [17-18 FINAL]

1. Synchronous vs Asynchronous Communication

Synchronous Communication

In synchronous communication, the sender and receiver must synchronize with each other. The sender waits until the receiver receives the message.

Benefits

- Simple and reliable
- Easier synchronization
- Ensures message delivery before continuing

Disadvantages

- Sender may block and wait

- Lower performance if receiver is slow
-

Asynchronous Communication

In asynchronous communication, the sender sends the message and continues execution without waiting.

Benefits

- Better performance
- Sender and receiver work independently
- Higher concurrency

Disadvantages

- More complex implementation
 - Possible message loss or overflow problems
-

2. Automatic vs Explicit Buffering

Automatic Buffering

The operating system automatically creates and manages communication buffers.

Automatic buffering-এ operating system নিজে থেকেই communication buffer তৈরি ও manage করে। Programmer-কে buffer manually handle করতে হয় না।

Benefits

- Easier for programmers
- Reduces programming complexity

Disadvantages

- Less control over memory usage
- May waste system resources

Example

ধরো WhatsApp message send করার সময় app নিজেই temporary buffer manage করছে। User-কে কিছু করতে হয় না।

Explicit Buffering

The programmer explicitly manages the buffer.

Explicit buffering-এ programmer নিজে buffer তৈরি ও manage করে।

Benefits

- Better control over memory and performance
- Efficient resource utilization

Disadvantages

- More programming complexity
- Greater chance of programming errors

Example

C programming-এ programmer নিজে array বা buffer declare করে data handle করছে।

Automatic vs Explicit Buffering(NTK)

বিষয়	Automatic Buffering	Explicit Buffering
Buffer কে manage করে	Operating System	Programmer
Complexity	কম	বেশি
Control	কম	বেশি
Error possibility	কম	বেশি

3. Send by Copy vs Send by Reference

Send by Copy

The actual data is copied from sender to receiver.

Benefits

- Safer communication
- Receiver cannot accidentally modify sender's data

Disadvantages

- Slower for large data
- Extra memory required

Example

একটি file-এর duplicate copy অন্য process-এ পাঠানো।

Send by Reference

Only the memory address (reference) of data is sent.

এখানে actual data পাঠানো হয় না, শুধু data-এর memory address/reference পাঠানো হয়।

Benefits

- Faster communication
- Less memory usage

Disadvantages

- Shared data may be modified accidentally
- Synchronization problems may occur

Example

দুইটি process একই shared memory ব্যবহার করছে।

Send by Copy vs Send by Reference(NTK)

বিষয়	Send by Copy	Send by Reference
কী পাঠানো হয়	Actual data	Memory reference
Speed	ধীর	দ্রুত
Memory Usage	বেশি	কম
Safety	বেশি	কম

4. Fixed-Sized vs Variable-Sized Messages

Fixed-Sized Messages

All messages have the same predefined size.

সব message-এর size আগে থেকেই নির্দিষ্ট থাকে।

Benefits

- Simple implementation
- Easier memory management
- Faster processing

Disadvantages

- Memory wastage may occur
- Not flexible for different data sizes

Example

প্রতিটি message সবসময় 128 bytes।

Variable-Sized Messages

Messages can have different sizes.

Benefits

- Flexible communication
- Efficient memory usage for varying data

Disadvantages

- More complex implementation
- Harder memory management

Example

Email message যেখানে প্রতিবার message size আলাদা হয়।

Fixed vs Variable Sized Message(NTK)

বিষয়	Fixed-Sized	Variable-Sized
Message Size	নির্দিষ্ট	পরিবর্তনশীল
Flexibility	কম	বেশি
Memory Management	সহজ	জটিল
Memory Waste	বেশি হতে পারে	কম

Conclusion

Each communication method has advantages and disadvantages. Synchronous communication is simple but slower, while asynchronous communication improves concurrency. Automatic buffering is easier, explicit buffering gives more control, send by copy is safer, send by reference is faster, fixed-sized messages are simple, and variable-sized messages are more flexible.